Cluster Multi-Master

Montando um cluster produtivo do zero com kubeadm e HAProxy



Ada · DE-OP-011 Orquestração · Kubernetes | Aula 08

Cronograma das Aulas

01
17/07 · Sexta
Conhecendo o Kubernetes

02
20/07 · Segunda
Mão na Massa: Minikube e kubectl

03
22/07 · Quarta
Pods, Namespaces e Controladores

04
24/07 · Sexta
Primeira Aplicação no K8S

05
27/07 · Segunda
Estratégias de Deploy e Limitação de Recursos

06
29/07 · Quarta
Services, Ingress e Volumes

07
31/07 · Sexta
Componentes e Troubleshooting

08
03/08 · Segunda
Cluster Multi-Master

09
05/08 · Quarta
Concluir atividades do módulo e preencher rubricas

COMBINADO DA TURMA

Por favor!

Bora participar!



Câmera ligada + mão levantada = aula top!

INTERVALO • 16:30 BRT

Combinados da nossa aula

1. Câmeras ligadas.
2. Entrem no horário: início e intervalo.
3. Participem da aula.
4. Tirem dúvidas.
5. Levantem a mão para tirar dúvidas.

TRABALHO FINAL

Entrega Final!

O desafio

- Suba no **minikube** dois Pods um com a API e o outro com o Banco de Dados à sua escolha.
- A API deve fazer um **CRUD para produtos**.

TABELA produtos

Id – INT

Nome – VARCHAR(50)

O que significa CRUD

- **C** — **Create** — Criar
- **R** — **Read** — Ler
- **U** — **Update** — Atualizar
- **D** — **Delete** — Excluir

Atualize a versão do Pod de forma **declarativa**, demonstrando:

Rolling Update **Recreate**

Objetivos de hoje

- Entender por que produção pede múltiplos masters
- Provisionar máquinas com Vagrant + VirtualBox
- Instalar os componentes com kubeadm
- Configurar o Load Balancer (HAProxy)
- Formar o cluster e conhecer clusters gerenciados (EKS/GKE/AKS)

Por que Multi-Master?

- Minikube = 1 node, ótimo para estudo, inaceitável para produção
- Um único master é ponto único de falha (SPOF)
- Múltiplos masters = alta disponibilidade do control plane
- Cada master tem sua instância do etcd
- Um Load Balancer centraliza o acesso à API

Pré-requisitos e topologia

- 4GB de RAM livres (4 máquinas virtuais)
- VirtualBox — virtualizador das máquinas
- Vagrant — cria e gerencia as VMs de forma automatizada
- Topologia: 3 masters + 1 balancer (HAProxy)
- Rede privada 172.27.11.x

1. Provisionando as máquinas

Vagrant + VirtualBox

Vagrantfile: as 4 máquinas como código

Vangrantfile - define balancer + 3 masters com IPs fixos

```
vms = {
  'balancer' => {'memory' => '2048', 'cpus' => 2, 'ip' => '200'},
  'master3'  => {'memory' => '2048', 'cpus' => 2, 'ip' => '30'},
  'master2'  => {'memory' => '2048', 'cpus' => 2, 'ip' => '20'},
  'master1'  => {'memory' => '2048', 'cpus' => 2, 'ip' => '10'}
}

Vagrant.configure('2') do |config|
  config.vm.box = 'debian/buster64'
  vms.each do |name, conf|
    config.vm.define name do |k|
      k.vm.hostname = "#{name}.example.com"
      k.vm.network 'private_network', ip: "172.27.11.#{conf['ip']}"
      k.vm.provider 'virtualbox' do |vb|
        vb.memory = conf['memory']
        vb.cpus = conf['cpus']
      end
    end
  end
end
```

Subindo e acessando as VMs

MÃO NA MASSA

Dentro do diretório k8s, suba as 4 máquinas:

```
$ vagrant up
```

Verifique o status:

```
$ vagrant status
```

Acesse uma máquina via SSH:

```
$ vagrant ssh master1
```

Para sair de uma máquina, pressione Ctrl+D

Subindo e acessando as VMs

```
PS C:\Users\TI\Documents\k8s> vagrant up
Bringing machine 'balancer' up with 'virtualbox' provider...
Bringing machine 'master3' up with 'virtualbox' provider...
Bringing machine 'master2' up with 'virtualbox' provider...
Bringing machine 'master1' up with 'virtualbox' provider...
==> balancer: Box 'debian/buster64' could not be found. Attempting to find and install...
    balancer: Box Provider: virtualbox
    balancer: Box Version: >= 0
==> balancer: Loading metadata for box 'debian/buster64'
    balancer: URL: https://vagrantcloud.com/debian/buster64
==> balancer: Adding box 'debian/buster64' (v10.20211018.1) for provider: virtualbox
    balancer: Downloading: https://vagrantcloud.com/debian/boxes/buster64/versions/10.20211018.1/providers/virtualbox.box
    balancer:
==> balancer: Successfully added box 'debian/buster64' (v10.20211018.1) for 'virtualbox'!
==> balancer: Importing base box 'debian/buster64'...
==> balancer: Matching MAC address for NAT networking...
==> balancer: Setting the name of the VM: k8s_balancer_1656644170487_74995
==> balancer: Clearing any previously set network interfaces...
==> balancer: Preparing network interfaces based on configuration...
    balancer: Adapter 1: nat
    balancer: Adapter 2: hostonly
==> balancer: Forwarding ports...
    balancer: 22 (guest) => 2222 (host) (adapter 1)
==> balancer: Running 'pre-boot' VM customizations...
==> balancer: Booting VM...
==> balancer: Waiting for machine to boot. This may take a few minutes...
    balancer: SSH address: 127.0.0.1:2222
    balancer: SSH username: vagrant
    balancer: SSH auth method: private key
```

Fonte: Let's Code

```
PS C:\Users\TI\Documents\k8s> vagrant status
Current machine states:

balancer                  running (virtualbox)
master3                   running (virtualbox)
master2                   running (virtualbox)
master1                   running (virtualbox)
```

Fonte da imagem: Ada Tech

Subindo e acessando as VMs (cont.)

```
PS C:\Users\TI\Documents\k8s> vagrant ssh master1
Linux master1 4.19.0-18-amd64 #1 SMP Debian 4.19.208-1 (2021-09-29) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
vagrant@master1:~$
```

Fonte da imagem: Ada Tech

2. Instalando componentes

Docker, kubelet, kubeadm, kubectl

Preparação de cada master

- Ajustar iptables para o modo legacy (compatibilidade com kube-proxy)
- Instalar Docker + kubelet + kubeadm + kubectl
- Desabilitar o swap (o Kubernetes exige)
- Configurar o cgroupdriver do Docker para systemd
- Fixar o node-ip do kubelet (várias interfaces no Vagrant)

Script de instalação (por master)

instala Docker e os componentes do Kubernetes

```
apt-get update
apt-get install -y apt-transport-https ca-certificates \
  curl gnupg2 software-properties-common vim
# repositório do Docker
curl -fsSL https://download.docker.com/linux/debian/gpg | apt-key add -
add-apt-repository "deb [arch=amd64] \
  https://download.docker.com/linux/debian $(lsb_release -cs) stable"
# repositório do Kubernetes
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | apt-key add -
echo 'deb https://apt.kubernetes.io/ kubernetes-xenial main' \
  > /etc/apt/sources.list.d/kubernetes.list
apt-get update
apt-get install -y docker-ce docker-ce-cli containerd.io \
  kubelet kubeadm kubectl
apt-mark hold kubelet kubeadm kubectl   # trava as versões
```

Script de instalação (por master)

```
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
root@master1:/home/vagrant#
```

Fonte da imagem: Ada Tech

3. Load Balancer e formação do cluster

HAProxy + kubeadm init/join

HAProxy: balanceando os 3 masters

/etc/haproxy/haproxy.cfg — porta 6443 (API)

```
frontend kubernetes
  bind 172.27.11.200:6443
  mode tcp
  default_backend kubernetes-master-nodes

backend kubernetes-master-nodes
  mode tcp
  balance roundrobin
  option tcp-check
  server k8s-master-0 172.27.11.10:6443 check fall 3 rise 2
  server k8s-master-1 172.27.11.20:6443 check fall 3 rise 2
  server k8s-master-2 172.27.11.30:6443 check fall 3 rise 2
```

HAProxy: balanceando os 3 masters

```
root@balancer:/home/vagrant#     ss -nltp | grep 6443 # Procura pelas portas TCP abertas
LISTEN    0         128          172.27.11.200:6443             0.0.0.0:*        users:(("haproxy",pid=2585,fd=4))
```

Fonte da imagem: Ada Tech

Iniciando o primeiro master

kubeadm-config.yml aponta para o Load Balancer

```
# /root/kubeadm-config.yml
apiVersion: kubeadm.k8s.io/v1beta2
kind: ClusterConfiguration
kubernetesVersion: stable
controlPlaneEndpoint: "172.27.11.200:6443"  # o HAProxy
networking:
  podSubnet: "10.244.0.0/16"
---
apiVersion: kubeadm.k8s.io/v1beta2
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: "172.27.11.10"
  bindPort: 6443

# inicia o cluster e emite os certificados compartilhados
# kubeadm init --config '/root/kubeadm-config.yml' --upload-certs
```

Iniciando o primeiro master

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

You can now join any number of the control-plane node running the following command on each as root:

  kubeadm join 172.27.11.200:6443 --token v1e8v4.2t5knz1l2k8obqzd \
        --discovery-token-ca-cert-hash sha256:44a93ccf4c1933c74491073fd0d7f84460d9d8324716acefec333277eec26c2c \
        --control-plane --certificate-key 4b8dd45345dbe050a82b878a6e5a612f81aae0e56c12a0fb192c016137506627

Please note that the certificate-key gives access to cluster sensitive data, keep it secret!
As a safeguard, uploaded-certs will be deleted in two hours; If necessary, you can use
"kubeadm init phase upload-certs --upload-certs" to reload certs afterward.

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 172.27.11.200:6443 --token v1e8v4.2t5knz1l2k8obqzd \
        --discovery-token-ca-cert-hash sha256:44a93ccf4c1933c74491073fd0d7f84460d9d8324716acefec333277eec26c2c
```

Fonte da imagem: Ada Tech

Rede (Calico) e juntando os masters

MÃO NA MASSA

No master1, configure o kubectl do root:

```
$ mkdir -p /root/.kube && cp /etc/kubernetes/admin.conf /root/.kube/config
```

Nodes ficam NotReady até instalar a rede. Instale o Calico:

```
$ kubectl apply -f calico.yaml
```

Agora o master1 fica Ready. Junte os demais masters (comando do init):

```
$ kubeadm join 172.27.11.200:6443 --token ... --control-plane --certificate-key ... --apiserver-advertise-address=172.27.11.20
```

Valide o cluster completo:

```
$ kubectl get nodes
```

Rede (Calico) e juntando os masters

```
root@master1:/home/vagrant# kubectl get nodes
NAME      STATUS     ROLES           AGE     VERSION
master1   NotReady   control-plane   3m26s   v1.24.2
```

Fonte da imagem: Ada Tech

```
root@master1:~# kubectl get nodes
NAME      STATUS   ROLES           AGE   VERSION
master1   Ready    control-plane   19m   v1.24.2
```

Fonte da imagem: Ada Tech

Validando com um DaemonSet

MÃO NA MASSA

Permita pods de aplicação nos masters (remove o taint padrão):

```
$ kubectl taint nodes --all node-role.kubernetes.io/master-
```

Aplique um DaemonSet (um Pod por máquina):

```
$ kubectl apply -f daemonset-sh-cgi.yaml
```

Confirme um Pod em cada master:

```
$ kubectl get pods -o wide
```

Clusters Gerenciados: EKS, GKE, AKS

- As clouds oferecem Kubernetes gerenciado — sem cuidar dos masters
- EKS (AWS) — o mais utilizado atualmente
- GKE (Google Cloud) — mais opções de gerenciamento e automação
- AKS (Azure) — boa relação custo-benefício
- Você foca nas aplicações; a cloud cuida do control plane e upgrades

Encerrando o módulo!

- ✓ Você foi de 'o que é container' a um cluster multi-master HA
- ✓ Domina Pods, controladores, Services, volumes, recursos e RBAC
- ✓ Sabe fazer deploy com estratégia e reverter com segurança
- ✓ Sabe investigar e resolver problemas (troubleshooting)
- ✓ Próximos passos: certificações (CKA/CKAD) e um cluster gerenciado

